

Tokenización Avanzada

Módulo 5 · NLP y Modelos de Lenguaje (LLMs)

BPE, WordPiece, SentencePiece, coste por idioma, tokens especiales y optimización de prompts para reducir coste de inferencia.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Tokenización Avanzada

El módulo 1 introdujo la tokenización como concepto. Este tema la profundiza con las implicaciones prácticas que más importan en producción: por qué el mismo contenido en diferentes idiomas tiene costes muy distintos, cómo la tokenización explica los fallos de los LLMs con números y caracteres especiales, qué diferencia hay entre el tokenizador de GPT-4, Llama 3 y BERT, y cómo la elección del tokenizador afecta al rendimiento del modelo en tu dominio específico.

En 2025, todos los LLMs de producción usan alguna variante de tokenización de subpalabras (Byte-Pair Encoding, WordPiece o SentencePiece). La tokenización es la interfaz entre el texto humano y el espacio numérico del modelo. Optimizar esta interfaz —elegir el tokenizador correcto, entender sus propiedades y gestionar sus limitaciones— tiene impacto directo en el coste, la calidad y el comportamiento de cualquier sistema de LLM en producción.

Los algoritmos de tokenización y sus propiedades

BPE (Byte-Pair Encoding): el estándar en GPT

BPE construye su vocabulario iterativamente: empieza con todos los bytes individuales (vocabulario base de 256 símbolos en byte-level BPE) y aplica repetidamente la fusión del par de tokens más frecuente en el corpus de entrenamiento. GPT-2 aprendió 50.000 fusiones (vocabulario ~50K tokens). GPT-4 usa `cl100k_base` con ~100K tokens. GPT-4o usa `o200k_base` con ~200K tokens. A mayor vocabulario, secuencias más cortas para el mismo contenido, pero mayor coste de la capa de embeddings. GPT-4 usa byte-level BPE, garantizando que cualquier carácter Unicode puede tokenizarse sin tokens desconocidos.

WordPiece: la variante de BERT

WordPiece (usado en BERT) es similar a BPE pero elige las fusiones que maximizan la probabilidad del corpus de entrenamiento bajo un modelo de lenguaje de unigramas, no simplemente la frecuencia del par. En la práctica, produce tokenizaciones muy similares a BPE. Una diferencia visible: WordPiece usa el prefijo "##" para marcar los tokens que son continuaciones de una palabra (no el inicio), mientras que BPE usa el carácter especial "▀" (para marcar el inicio de una palabra precedida de espacio). BERT tiene un vocabulario de 30.522 tokens para la versión inglesa.

SentencePiece: tokenización independiente del idioma

SentencePiece (Google) tokeniza directamente desde el texto crudo sin pre-tokenización por espacios, tratando el texto como una secuencia de caracteres Unicode. Esto lo hace especialmente adecuado para idiomas sin separación explícita de palabras (chino, japonés, tailandés) y para tokenización multilingüe. Llama 2 y algunos modelos multilingües usan SentencePiece con el algoritmo BPE o Unigram Language Model.

Los efectos prácticos de la tokenización en el comportamiento del modelo

Por qué los LLMs fallan en operaciones sobre caracteres

La pregunta "¿cuántas letras r hay en strawberry?" produce respuestas incorrectas en muchos LLMs porque "strawberry" puede tokenizarse como un único token. El modelo ve el ID numérico del token, no los caracteres individuales que lo componen. Si el token "strawberry" corresponde al ID 12345, el modelo no tiene acceso directo a los caracteres s-t-r-a-w-b-e-r-r-y a menos que razone explícitamente sobre la composición del token, lo que requiere capacidades de razonamiento extendido.

Tokenización y coste de idiomas

Los tokenizadores BPE aprenden sus fusiones a partir del corpus de entrenamiento. Si ese corpus está dominado por texto en inglés (como es el caso de la mayoría de los LLMs occidentales), las palabras frecuentes en inglés se fusionan en tokens únicos mientras que palabras frecuentes en otros idiomas se fragmentan en más tokens. Chino y árabe son especialmente caros: una frase corta en chino puede producir el doble de tokens que en inglés para el mismo contenido semántico.

Tokens especiales y el formato de conversación

Además de los tokens de texto, los LLMs usan tokens especiales para estructurar el formato de entrada. Los modelos instruct usan tokens especiales como <|system|>, <|user|>, <|assistant|> (Llama), [INST] y [/INST] (Mistral), o <|im_start|> y <|im_end|> (formato ChatML). Estos tokens especiales son críticos: enviar un prompt al modelo instruct sin los tokens de formato correctos produce resultados de calidad degradada o completamente incorrectos.

Tokenización multilingüe y modelos especializados

Para aplicaciones multilingües, los modelos con tokenizadores entrenados con más balance entre idiomas producen mejores resultados y menor coste en idiomas no ingleses. Gemma de Google y Aya de Cohere están específicamente diseñados para tokenización multilingüe más eficiente. Para dominios muy especializados (código en un lenguaje poco frecuente, texto médico con mucha nomenclatura latina), puede valer la pena entrenar un tokenizador propio sobre corpus del dominio para reducir el número de tokens por carácter.

SECCIÓN 4 · EJEMPLOS REALES

- Impacto de la tokenización en costes de API para un portal de noticias en español: el sistema procesa 500.000 artículos de noticias al mes en español para clasificación y resumen. Con GPT-4 (cl100k_base), el coste es un 20% mayor que si los artículos fueran en inglés del mismo contenido. Usando un modelo con tokenizador más eficiente para español (Mistral o un modelo multilingüe) reduce el número de tokens y el coste proporcionalmente.
- Diagnóstico de fallos de aritmética: un sistema de análisis financiero descubrió que el LLM fallaba al calcular variaciones porcentuales con números de muchos dígitos. La causa era la tokenización: "1234567.89" se tokenizaba en varios tokens, y el modelo no procesaba el valor numérico completo de forma cohesiva. La solución: conectar el LLM a un intérprete de

Python para todos los cálculos numéricos.

- Tokens especiales en formato de conversación: un desarrollador que migró de GPT-4 a Llama 3 Instruct descubrió que el rendimiento caía dramáticamente. La causa: el código no aplicaba el template de conversación correcto de Llama 3 (que usa tokens especiales específicos). Una vez aplicado el template correcto, el rendimiento se recuperó completamente.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Estimar tokens como palabras. Un token no es una palabra: en inglés, 1 token $\approx$ 0.75 palabras; en español, 1 token $\approx$ 0.65 palabras; en código, la ratio varía enormemente. Usa tiktoken (para modelos OpenAI) o el tokenizador específico del modelo para calcular exactamente los tokens de cualquier texto antes de hacer estimaciones de coste.

Error 2: Ignorar los tokens especiales al calcular el contexto disponible. En un modelo con ventana de 128K tokens, los tokens de sistema, los tokens del historial de conversación, y los tokens especiales de formato consumen parte de ese espacio. El contexto real disponible para el documento a procesar puede ser significativamente menor que la ventana total.

Error 3: No verificar el template de conversación al cambiar de modelo. Cada familia de modelos instruct tiene un formato de conversación específico con tokens especiales distintos. Cambiar de modelo sin actualizar el template de conversación es uno de los errores más frecuentes en migración de sistemas LLM y produce degradaciones silenciosas de rendimiento.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Herramientas de tokenización para uso práctico: tiktoken (pip install tiktoken) para todos los modelos de OpenAI; tokenizers de Hugging Face (pip install transformers) para prácticamente cualquier otro modelo. Para estimar tokens antes de enviar: `enc = tiktoken.encoding_for_model("gpt-4o")`, `tokens = enc.encode(texto)`, `print(len(tokens))`.

Para sistemas multilingüe con sensibilidad al coste: establece un benchmark de tokenización que mida el número de tokens por carácter para los idiomas que tu sistema necesita procesar. Compara los modelos candidatos en esta métrica. Un modelo con 20% menos tokens para tu idioma principal es un 20% más barato en inferencia, todo lo demás igual.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M1-T10 (Tokenización básica): introduce BPE y el concepto de token. Este tema profundiza en las implicaciones prácticas avanzadas.
- M6 (Ingeniería de prompts): la optimización de tokens es una dimensión de la ingeniería de prompts que tiene impacto económico directo.
- M9 (Infraestructura): el coste de inferencia es directamente proporcional a los tokens procesados. La tokenización afecta al presupuesto de infraestructura.

SECCIÓN 8 · RESUMEN EJECUTIVO

La tokenización convierte texto en secuencias de IDs numéricos usando vocabularios de 30K-200K tokens aprendidos estadísticamente. BPE (GPT), WordPiece (BERT) y SentencePiece (Llama, modelos multilingües) son los algoritmos dominantes con propiedades ligeramente distintas. Los LLMs fallan en operaciones sobre caracteres porque pueden no tener acceso directo a los caracteres individuales dentro de un token. Los idiomas distintos del inglés son 15-30% más caros en tokens. Los tokens especiales de formato son críticos al cambiar de modelo. Herramientas prácticas: tiktoken para OpenAI, Hugging Face tokenizers para el resto. Optimización: formato compacto, chunking selectivo, eliminar espacios redundantes.